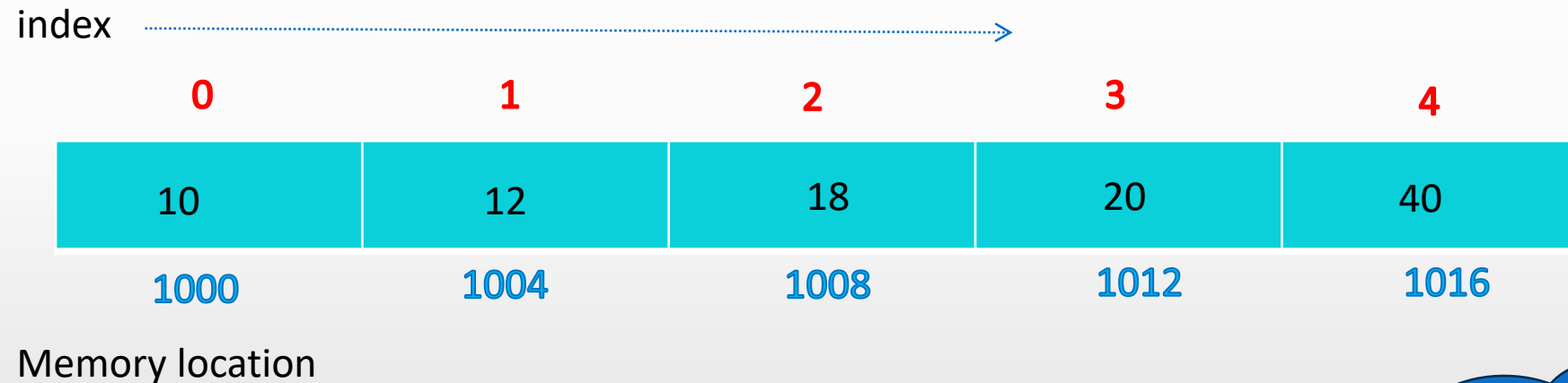


Introduction

An array is a linear data structure that collects elements of the same data type and stores them in contiguous and adjacent memory locations.

Arrays work on an index system starting from 0 to (n-1), where n is the size of the array.



We can directly access an array element by using its index value.

Basic terminologies of array

- **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
- **Array Length:** The length of an array is determined by the number of elements it can contain.
- **Array element:** Elements are items stored in an array and can be accessed by their index.

Syntax for array declaration

```
data_type array_name [array_size];
```

Example

```
float marks [15];
```

Declaration of an Array

Declaring arrays - specify:

Name

Type of array

Number of elements

Examples

```
int c[ 10 ];
```

```
float hi[ 3284 ];
```

Declaring multiple arrays of same type

Similar format as other variables

Example

```
int b[ 100 ], x[ 27 ];
```

Declaration of an Array

After an array is declared, its elements must be initialized. Array can be initialized by two ways:

- **At Compile time**

The general form of initialization of an array is

```
type array_name[size] = {list of values};
```

The values in the list are separated by commas.

For example,

```
int rollno [5]={213,214,215,216,217};
```

- **At Run Time**

An array can also be explicitly initialized at run time.

For example,

```
for(i=0,i<10,i++)  
{  
    scanf("%d",&a[i]);  
}
```

Declaration of an Array

What is the difference between the following two declarations ?

```
char codes[] = {'s', 'a', 'm', 'p', 'l', 'e'};
```

```
char codes[] = "sample";
```

```
char codes[] = {'s', 'a', 'm', 'p', 'l', 'e'};
```

In this declaration:

`char codes[]` declares a character array named `codes`. The size of the array is automatically determined based on the number of elements initialized.

`{'s', 'a', 'm', 'p', 'l', 'e'}` is the initializer list, which explicitly lists each character that will be stored in the array.

Here, each character ('s', 'a', 'm', 'p', 'l', 'e') is enclosed in single quotes ('). Single quotes denote individual characters in C and C++.

Declaration of an Array

```
char codes[] = "sample";
```

In this declaration:

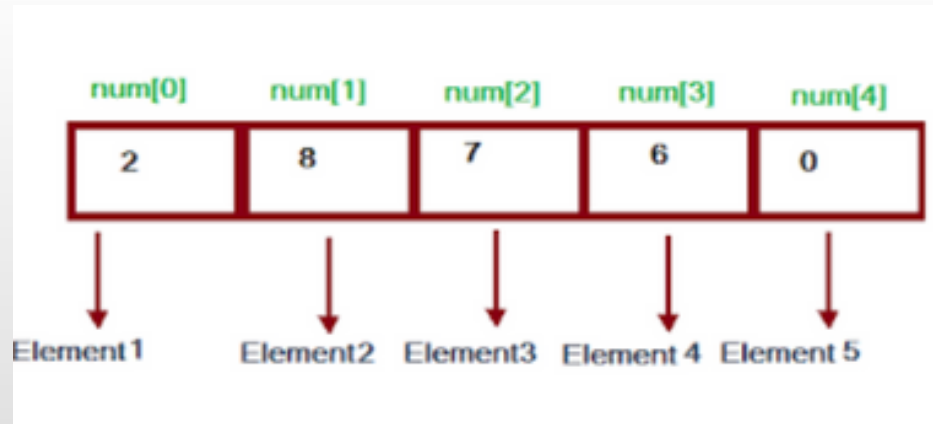
`char codes[]` declares a character array named `codes`. Similar to the first declaration, the size of the array is automatically determined based on the length of the string literal `"sample"`.

`"sample"` is a string literal in double quotes (`"`). In C and C++, double quotes denote string literals, which are null-terminated arrays of characters (i.e., strings). The string literal `"sample"` implicitly includes a null terminator (`'\0'`) at the end. Therefore, the array `codes` will have a size of 7 (6 characters + 1 null terminator).

Types of an Array

1. Single Dimensional Array

A single-dimensional array is the simplest type of array that stores elements in a linear sequence. It is also known as a one-dimensional array, and each element in the array can be accessed using a single index. The index ranges from 0 to the length of the array minus one.

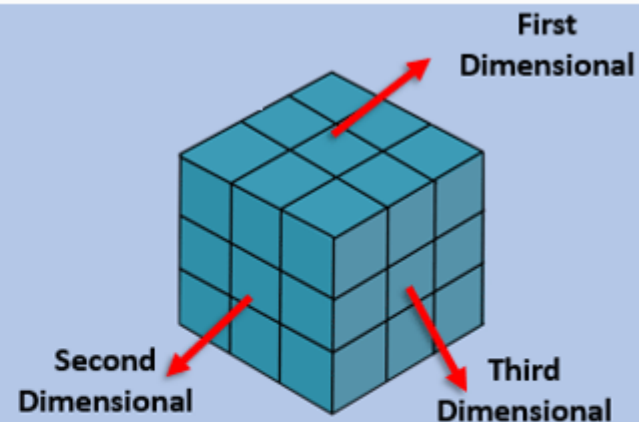


2. Multidimensional Array

A multi-dimensional array is an array that stores elements in a two-dimensional or higher-dimensional structure. It is also known as a two-dimensional or higher-dimensional array, and each element in the array can be accessed using multiple indices. Multi-dimensional arrays can be of different dimensions, including two-dimensional, three-dimensional, or higher-dimensional arrays.

	Column 0	Column 1	Column 2
Row 0	X[0][0]	X[0][1]	X[0][2]
Row 1	X[1][0]	X[1][1]	X[1][2]
Row 2	X[2][0]	X[2][1]	X[2][2]

2D-Array



3D-Array

Memory allocation of an array

All the data elements of an array are stored at contiguous locations in the main memory

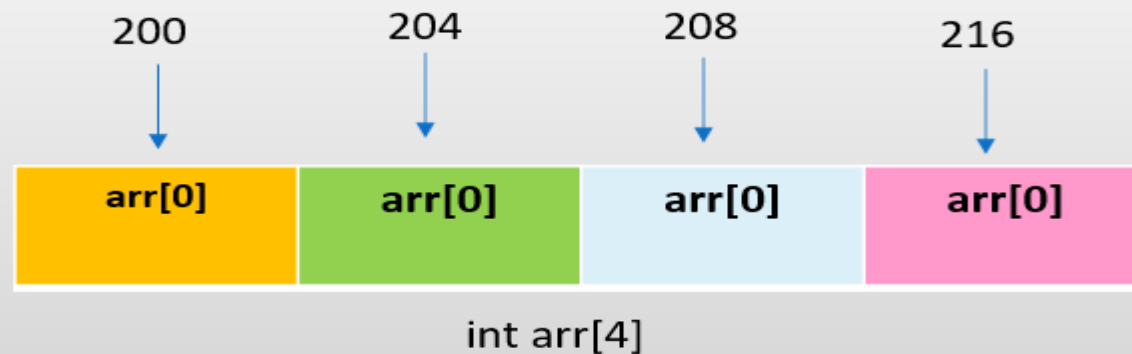
The name of the array represents the base address or the address of the first element in the main memory.

We can define the indexing of an array in the below ways -

0 (zero-based indexing): The first element of the array will be `arr[0]`.

1 (one-based indexing): The first element of the array will be `arr[1]`.

n (n - based indexing): The first element of the array can reside at any random index number.



How to access an element from the 1-D array?

the information given below to access any random element from the array -

- Base Address of the array.
- Size of an element in bytes.
- Type of indexing, array follows.

The formula to calculate the address to access an array element -

Byte address of element $A[i]$ $\simeq$ base address + size * (i - first index)

Here, size represents the memory taken by the primitive data types.

Example

- Given the base address of an array A[300 900] as 1004 and the size of each element is 2 bytes in the memory, find the address of A[500].

Base address B = 1004

First index = 300

Storage size of one element store in any array (size) = 2 Byte

Subset of element whose address to be found I = 500

Formula used:

Byte address of element A[i] = base address + size * (i - first index)

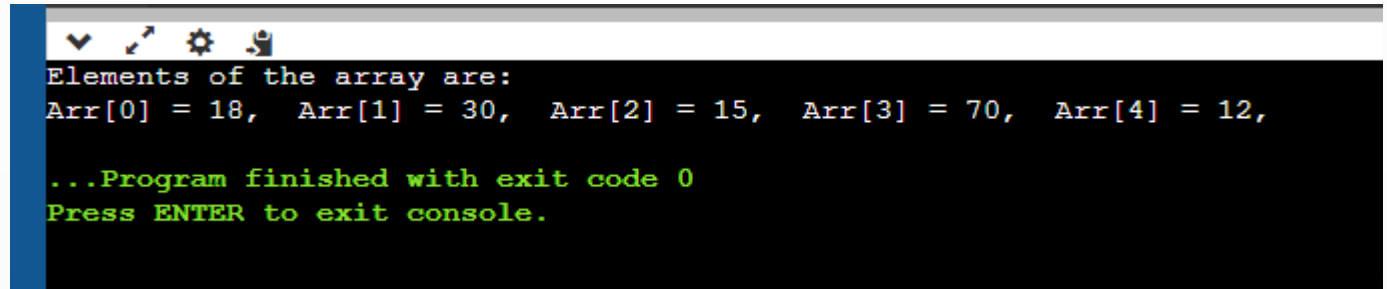
Solution:

$$\begin{aligned}\text{Address of A[500]} &= 1004 + 2 * (500 - 300) \\ &= 1004 + 2 * (200) \\ &= 1004 + 400\end{aligned}$$

Address of A[500] = 1404

Traversal - This operation is used to print the elements of the array.

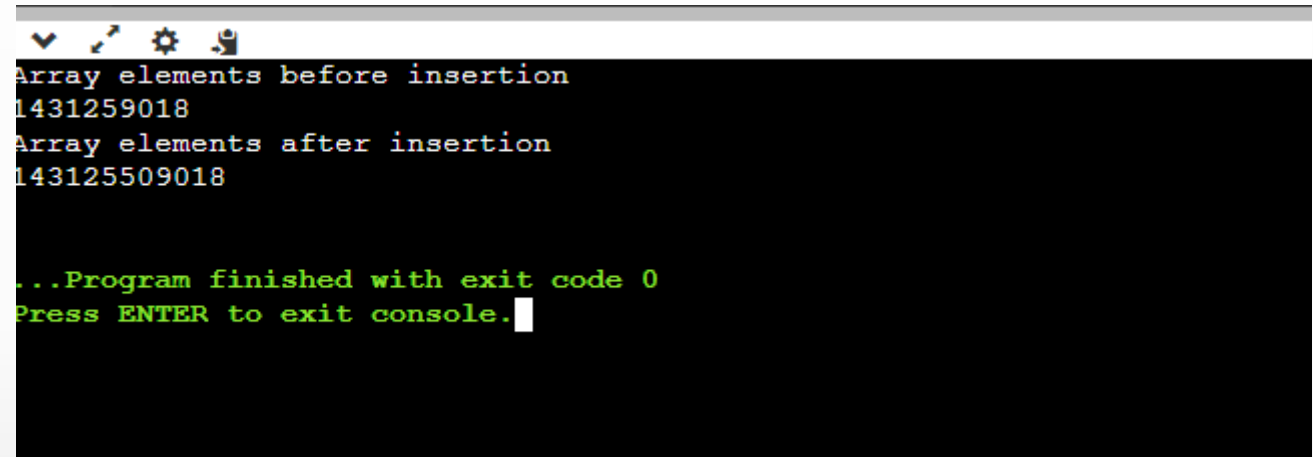
```
#include <stdio.h>
void main() {
    int Arr[5] = {18, 30, 15, 70, 12};
    int i;
    printf("Elements of the array are:\n");
    for(i = 0; i<5; i++) {
        printf("Arr[%d] = %d, ", i, Arr[i]);
    }
}
```



```
Elements of the array are:
Arr[0] = 18, Arr[1] = 30, Arr[2] = 15, Arr[3] = 70, Arr[4] = 12,
...Program finished with exit code 0
Press ENTER to exit console.
```

Insertion - It is used to add an element at a particular index.

```
#include<stdio.h>
int main()
{
    int arr[15] = { 14, 31, 25, 90, 18 };
    int i, x, pos, n = 5;
    printf("Array elements before insertion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    x = 50; // element to be inserted
    pos = 4;
    n++;
    for (i = n-1; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = x;
    printf("Array elements after insertion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

A screenshot of a console window showing the output of a C program. The window has a title bar with standard icons. The output text is as follows:
Array elements before insertion
1431259018
Array elements after insertion
143125509018

...Program finished with exit code 0
Press ENTER to exit console.
The text is displayed in a monospaced font with some color coding: the first two lines are in red, the next two in green, and the final two in blue.

```

#include<stdio.h>
int main()
{
    int arr[15] = { 14, 31, 25, 90, 18 };int main()
{
    int array[100],loc,size, value;

    printf("Enter number of elements in array");
    scanf("%d", &size);

    printf("Enter the elements");
    for(int i = 0; i < size; i++)
        scanf("%d", &array[i]);

    printf("Enter the position you want to insert in the array");
    scanf("%d", &loc);

    printf("Enter the value you want to insert at that position in the array");
    scanf("%d", &value);

    printf("shifting the elements from that location to size of array to right");
    for(int i = size-1; i >= loc-1; i--)
        array[i+1] = array[i];

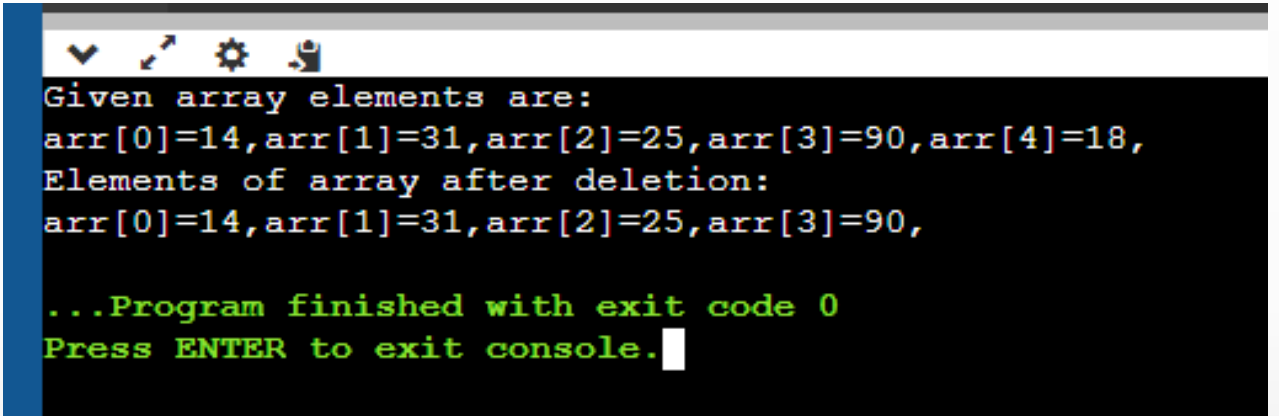
    array[loc-1] = value; // inserting the given value

    printf("Resultant array is: ");
    /*
        the array size gets increased by 1
        after insertion of the element
    */
    for(int i = 0; i <= size; i++)

```

Deletion - It is used to delete an element from a particular index.

```
#include <stdio.h>
void main() {
int arr[15] = { 14, 31, 25, 90, 18 };
int k = 90, n = 5;
int i, j;
printf("Given array elements are :\n");
for(i = 0; i<n; i++) {
    printf("arr[%d] = %d, ", i, arr[i]);
}
j = k;
while( j < n) {
    arr[j-1] = arr[j];
    j = j + 1;
}
n = n -1;
printf("\nElements of array after deletion:\n");
for(i = 0; i<n; i++) {
    printf("arr[%d] = %d, ", i, arr[i]);
}
}
```



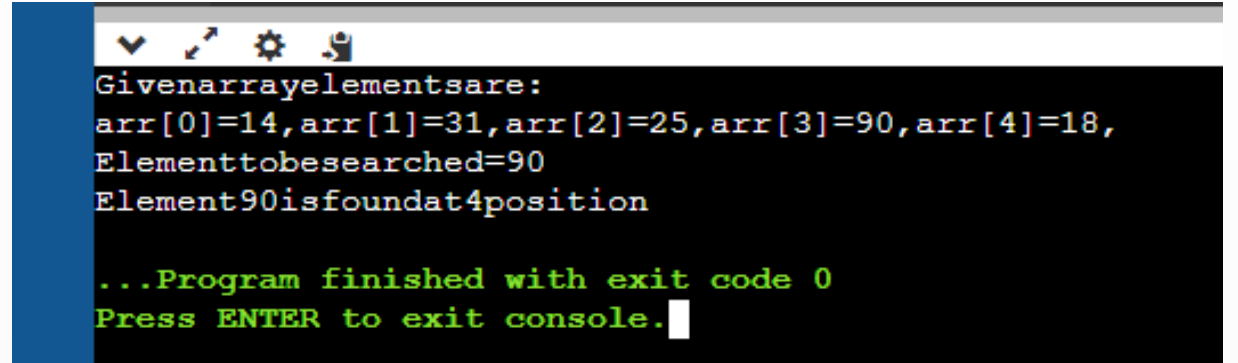
```
Given array elements are:
arr[0]=14,arr[1]=31,arr[2]=25,arr[3]=90,arr[4]=18,
Elements of array after deletion:
arr[0]=14,arr[1]=31,arr[2]=25,arr[3]=90,

...Program finished with exit code 0
Press ENTER to exit console.
```

Search - It is used to search an element using the given index or by the value.

```
#include <stdio.h>
void main() {
    int arr[5] = { 14, 31, 25, 90, 18 };
    int item = 90, i, j=0 ;
    printf("Given array elements are :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }
    printf("\nElement to be searched = %d", item);
    while( j < 5){
        if( arr[j] == item ) {
            break;
        }
        j = j + 1;
    }
    printf("\nElement %d is found at %d position", item, j+1);
}
```



```
Given array elements are:
arr[0]=14,arr[1]=31,arr[2]=25,arr[3]=90,arr[4]=18,
Element to be searched=90
Element 90 is found at 4 position

...Program finished with exit code 0
Press ENTER to exit console.
```


Update - It updates an element at a particular index.

```
#include <stdio.h>

void main() {
    int arr[5] = { 14, 31, 25, 90, 18 };
    int item = 50, i, pos = 3;

    printf("Given array elements are :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }

    arr[pos-1] = item;
    printf("\nArray elements after updation :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }
}
```

2 D Array

2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.

The syntax of declaring two dimensional array is very much similar to that of a one dimensional array, given as follows.

```
int arr[max_rows][max_columns];
```

	0	1	2		n-1
0	a[0][0]	a[0][1]	a[0][2]		a[0][n-1]
1	a[1][0]	a[1][1]	a[1][2]		a[1][n-1]
2	a[2][0]	a[2][1]	a[2][2]		a[2][n-1]
3	a[3][0]	a[3][1]	a[3][2]		a[3][n-1]
4	a[4][0]	a[4][1]	a[4][2]		a[4][n-1]
.	.	.	.		.
.	.	.	.		.
n-1	a[n-1][0]	a[n-1][1]	a[n-1][2]		a[n-1][n-1]

a[n][n]

we can access the individual cells in a 2D array by using the indices of the cells. There are two indices attached to a particular cell, one is its row number while the other is its column number.

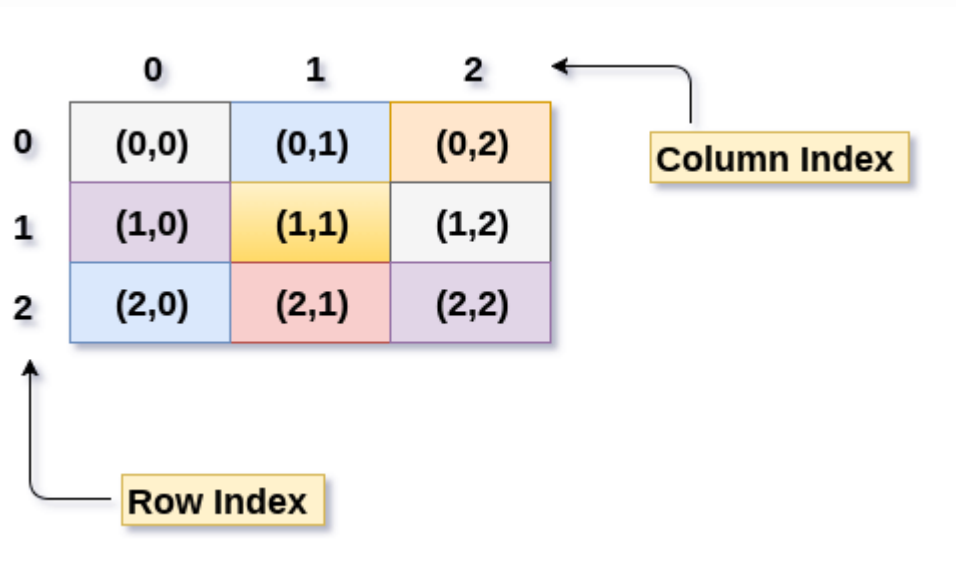
However, we can store the value stored in any particular cell of a 2D array to some variable x by using the following syntax.

```
int x = a[i][j];
```

where i and j is the row and column number of the cell respectively.

Mapping 2D array to 1D array

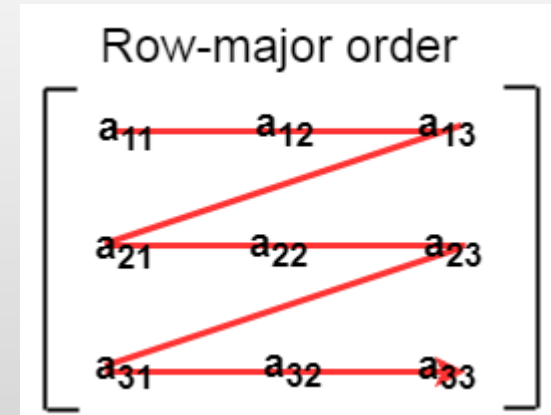
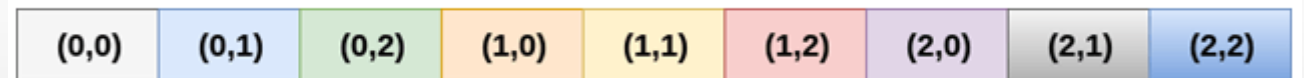
The size of a two dimensional array is equal to the multiplication of number of rows and the number of columns present in the array



There are two main techniques of storing 2D array elements into memory

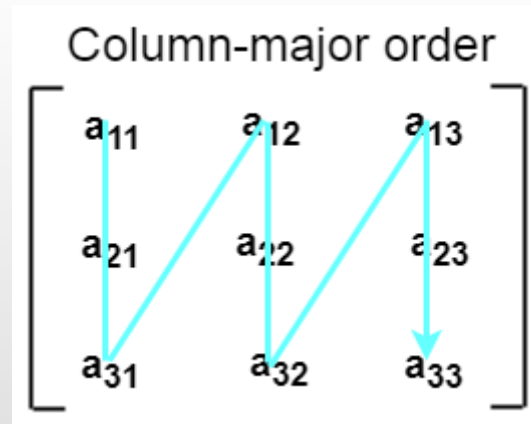
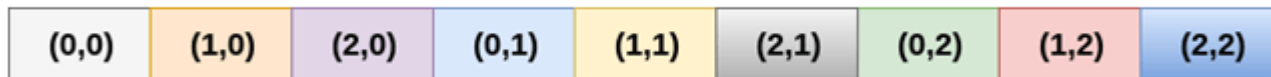
Row Major ordering

In row major ordering, all the rows of the 2D array are stored into the memory contiguously.



Column Major ordering

According to the column major ordering, all the columns of the 2D array are stored into the memory contiguously.



Calculating the Address of the random element of a 2D array

By Row Major Order

If array is declared by $a[m][n]$ where m is the number of rows while n is the number of columns, then address of an element $a[i][j]$ of the array stored in row major order is calculated as,

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

I = Row Subset of an element whose address to be found,

J = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in an array(in byte),

LR = Lower Limit of row/start row index of the matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of the matrix(If not given assume it as zero),

N = Number of column given in the matrix.

Example

Given an array **arr[10.....30][55.....75]** with a base address of **0** and the size of each element is **4 Bytes** in memory. Find the address of arr[15][58] with the help of Row-major order.

arr[10...30] [55...75], base address of the array (B) = 0,

size of an element = 4 bytes .

Find the location of a[15][58].

Address of A[I][J] = B + W * ((I - LR) * 20 + (J - LC))

Address(a[15][58]) = 0 + 4 * ((15 - 10) * 20 + (58 - 55))

= 0 + 4 * (5 * 20 + 3)

= 412

(10,55)	(10,56)	(10,57)	(10,58)	(10,59)	(10,60)		(10,75)
(11,55)	(11,56)	(11,57)	(11,58)	(11,59)	(11,60)		(11,75)
(12,55)	(12,6)	(12,57)	(12,58)	(12,59)	(12,60)		(12,75)
(13,55)	(13,56)	(13,57)	(13,58)	(13,59)	(13,60)		(13,75)
(14,55)	(14,56)	(14,57)	(14,58)	(14,59)	(14,60)		(14,75)
(15,55)	(15,56)	(15,57)	(15,58)	(15,59)	(15,60)		(15,75)
.	.	.	.	.	.		.
.	.	.	.	.	.		.
.	.	.	.	.	.		.
(30,55)	(30,56)	(30,57)	(30,58)	(30,59)	(30,60)		(30,75)

By Column Major Order

If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order. To find the address of the element using column-major order use the following formula:

$$\text{Address of } A[I][J] = B + W * ((J - LC) * M + (I - LR))$$

I = Row Subset of an element whose address to be found,

J = Column Subset of an element whose address to be found,

B = Base address,

W = Storage size of one element store in any array(in byte),

LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),

M = Number of rows given in the matrix.

Problem

1. Given the base address of an array **A[1300 1900]** as **1020** and the size of each element is 2 bytes in the memory, find the address of **A[1700]**.
2. Given an array, **arr[1.....10][1.....15]** with base value **100** and the size of each element is **1 Byte** in memory. Find the address of **arr[8][6]** with the help of row-major order.

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

Problem

3. Given an array **arr[1.....10][1.....15]** with a base value of **100** and the size of each element is **1 Byte** in memory find the address of **arr[8][6]** with the help of column-major order.

$$\text{Address of } A[I][J] = B + W * ((J - LC) * M + (I - LR))$$

Problem

4. : An array $P[35][14]$ is stored along the column with each of its elements occupying 8 bytes. Find out the base address and the address of an element $P[2][5]$, if the location $P[5][10]$ is stored at the address 4000 .
5. An array $Arr[50][30]$ is stored along the row with each of its elements occupying 4 bytes. Find out the base address and the address of an element $Arr[15][25]$, if the location $Arr[15][20]$ is stored at the address 8000.

Searching

Searching in data structure refers to the process of finding the required information from a collection of items stored as elements in the computer memory.

Searching in the data structure can be done by applying searching algorithms to check for or extract an element from any form of stored data structure.

These algorithms are classified according to the type of search operation they perform, such as:

- Sequential search**

The list or array of elements is traversed sequentially while checking every component of the set. For example – Linear Search.

- Interval Search**

The interval search includes algorithms that are explicitly designed for searching in sorted data structures. In terms of efficiency, these algorithms are far better than linear search algorithms. Example- Logarithmic Search, Binary search.

Thank You